# Measuring Semantic Preservation in Passkey Migration

*Anonymous Authors*

## Abstract

An imported passkey can still authenticate while a property needed by an application has changed. We present PasskeyTransit, a reproducible harness that measures identity, WebAuthn behavior, extension state, and operational guarantees separately. A balanced synthetic campaign exercised 6,144 credential–route–repetition attempts; 5,120 reached import and 1,024 were rejected before a ceremony. Every executed assertion authenticated, yet 512 imported attempts failed an executed browser behavior oracle for user verification or largeBlob. These fractions describe the constructed matrix, not product prevalence. Follow-up controls distinguish a project-authored CXP binding from selected provider behavior. A Chromium virtual-authenticator transfer preserved the signing credential but lost its PRF result; four public assertions from a replay were independently reverified. Fenko authenticated after CXF import while losing the original HMAC seeds. A Proton software authenticator preserved both seeds and reproduced the expected PRF with user verification. We provide claim boundaries and public verifiers. We did not establish public CXP wire interoperability between providers or a verified Android transfer.

## 1 Introduction

An accepted import and successful login are weak tests of passkey migration. A relying party can also depend on the credential identifier, WebAuthn extension output, user-verification behavior, and whether transfer retries leave duplicate state. The Credential Exchange Format (CXF) represents credentials [1]; the draft Credential Exchange Protocol (CXP) addresses protected exchange [2]. WebAuthn authenticates an assertion [5], but a valid signature does not by itself show that every credential property survived the move.

We model a credential as $C = (I, K, F, S)$: identity, signing material, observable functions, and operational guarantees. Our falsifiable proposition is that valid authentication does not imply preservation of every component. We contribute: (1) a claim matrix and property-specific oracles; (2) a synthetic campaign with explicit denominators and non-evaluable properties; (3) executable controls using two project-authored runtimes, Chromium virtual authenticators, and selected open-source provider code; and (4) an artifact whose public reports allow independent checks of retained signatures and PRF digest relationships. This is a measurement method with selected software cases, not a survey of deployed products.

## 2 Background and related work

CXF can encode FIDO2 HMAC state [1]. The CXP working draft defines a transport profile [2]; our single-document CXP/HPKE binding is an experimental project implementation and does not cover its full ZIP/JWE wire format [4]. HPKE follows RFC 9180 [3]. A WebAuthn PRF result is returned through client extension results, outside the ordinary signed assertion bytes [5]. Consequently, signature and PRF oracles answer separate questions. Ramat et al. study experiences across 111 websites [6]; Jannett et al. measure deployment [7]; Büttner and Gruschka compare device-bound and synced credentials [8]. Daffalla et al. study interpersonal abuse [9]; Islam et al. study cloud storage compromise [10]; Henarejos proposes authenticated device-bound restoration [11]. None measures whether a controlled credential retains extension behavior after a specific provider import.

Our unit is an attempted migration of a controlled credential, with separate post-import oracles for signing, extension behavior, and retry state. The method makes loss of observability an explicit outcome, rather than interpreting a successful assertion as evidence about unexecuted properties. The controlled routes expose that distinction, while the selected provider cases test only a few concrete code paths. We do not infer website adoption, attack prevalence, or provider prevalence.

## 3 Study design

### 3.1 Units, outcomes, and exclusions

The v1.5 protocol crossed 256 synthetic ES256 credentials (32 in each of eight strata) with 12 routes and two repetitions: 6,144 attempts in 96 route-by-stratum groups. The strata covered basic credentials, PRF with and without UV, largeBlob, credBlob, a payment-format marker, combined extensions, and a future optional member. They were constructed controls, not random draws. A route could reject the document before WebAuthn. For each applicable property we record PASS, FAIL, NOT_APPLICABLE, or NOT_EVALUABLE. The last outcome is needed where CDP cannot inject PRF or credBlob state.

One oracle checks CXF representation and policy. Each imported attempt performs two WebAuthn assertions: the first binds a challenge to the attempt, credential, route, repetition, public key, user handle, RP ID, and origin; the second binds its challenge to the first and to a canonical digest of extension observations. A separate verifier checks both ES256 signatures, the RP hash, user-presence and verification flags, and counter. Extension oracles examine executed behavior where the interface permits it; representation loss alone is never relabeled as an executed PRF failure. A deterministic in-memory simulation tests transfer atomicity and retry behavior. These oracles are not interchangeable. Campaign percentages are design-weighted proportions, without population confidence intervals; paired repetitions are stability checks, not independent providers. The browser and provider follow-ups were designed after inspecting earlier results and are exploratory rather than preregistered confirmations.

### 3.2 Threat and implementation boundary

Two separately written project implementations exchange one CXF document in Python-to-Node and Node-to-Python directions and reject a modified ciphertext. This tests that experimental binding. Exporter identity, authorization, durable provider storage, and the public CXP ZIP/JWE envelope are outside the tested profile. The transaction model is in memory and makes no claim about crash durability.

The reference route models an importer that retains supported fields. The strict route refuses a credential when it cannot preserve a required property. The compatible-lossy route accepts a credential while dropping selected extension state; the legacy route has a different loss policy. These are controlled policies written for this study, not named vendors. Direct, round-trip, and multihop chains expose whether a later capable destination can recover a field already lost at an earlier hop.

The source credential, expected CXF fields, and route policy are known to the harness. Its representation oracle therefore has ground truth that a real RP usually lacks. The browser oracle sees only what a WebAuthn ceremony and client extension results expose. This separation prevents a missing serialized seed from being counted as a failed browser PRF evaluation. The same distinction applies to a payment-format marker without a Secure Payment Confirmation ceremony.

Table 1: C1 synthetic attempts; denominator 6,144.

| Classification | Count | Share |
|---|---|---|
| Complete observable PASS | 2,304 | 37.50% |
| Visible degradation | 960 | 15.62% |
| Silent degradation | 1,344 | 21.88% |
| Not evaluable | 512 | 8.33% |
| Pre-ceremony rejection | 1,024 | 16.67% |

## 4 Synthetic campaign results

Of 6,144 attempts, 1,024 were rejected before ceremony and 5,120 were imported. Every executed assertion among the imported attempts was accepted. Table 1 separates those accepted assertions from other oracles. The complete observable PASS yield was 2,304/6,144 (37.50%). This describes the balanced test design, not the probability that an arbitrary user or product will succeed.

Among imported attempts, 512 (10.00%) combined valid login with failure of an executed UV or largeBlob browser oracle. PRF or credBlob representation loss occurred in 1,792 (35.00%); this category overlaps the former and does not establish an executed PRF failure in those cells. The union of non-payment failures was 2,048/5,120 (40.00%). A payment-format marker raises the design sensitivity to 2,304/5,120 (45.00%), but is not a Secure Payment Confirmation ceremony. C2 malformed document controls comprised 80 cases across ten families and eight strata; 64 were rejected by structural validation, duplicate policy, or strict preservation. C3 exercised 960 in-memory failure sequences and 1,920 events: 832 rolled back and converged to one copy on retry, while the 128 deliberate legacy late-failure controls retained a provisional and duplicated on retry. Neither C2 nor C3 is included in the C1 percentages.

Table 2 shows the route dependence directly. R03, R06, and R09 each contain 384 silent-degradation attempts, despite 512 imports and valid assertions. In the paired R02/R05 comparison, returning to the reference provider leaves the same 256 visible-degradation attempts: a later capable endpoint cannot reconstruct dropped source state. Strict handling after a lossy hop rejects 256 attempts in R08; after a legacy hop it rejects 384 in R10. These are consequences of the specified control policies, not failure rates for real providers.

Table 3 separates which controls generate the aggregate. The basic and optional-member strata each have 768 observable PASS attempts. PRF-bearing and credBlob strata have

Table 2: C1 outcomes by route. Each route has 512 designed attempts. P: complete observable PASS; V: visible degradation; S: silent degradation; NE: not evaluable. Counts are exact for this design.

| Route | Chain | Imported | Rejected | P | V | S | NE |
|---|---|---|---|---|---|---|---|
| R01 | ref–strict | 512 | 0 | 256 | 0 | 0 | 256 |
| R02 | ref–lossy | 512 | 0 | 256 | 256 | 0 | 0 |
| R03 | ref–legacy | 512 | 0 | 128 | 0 | 384 | 0 |
| R04 | ref–strict–ref | 512 | 0 | 256 | 0 | 0 | 256 |
| R05 | ref–lossy–ref | 512 | 0 | 256 | 256 | 0 | 0 |
| R06 | ref–legacy–ref | 512 | 0 | 128 | 0 | 384 | 0 |
| R07 | ref–strict–lossy | 512 | 0 | 256 | 256 | 0 | 0 |
| R08 | ref–lossy–strict | 256 | 256 | 256 | 0 | 0 | 0 |
| R09 | ref–strict–legacy | 512 | 0 | 128 | 0 | 384 | 0 |
| R10 | ref–legacy–strict | 128 | 384 | 128 | 0 | 0 | 0 |
| R11 | ref–lossy–legacy–strict | 128 | 384 | 128 | 0 | 0 | 0 |
| R12 | ref–strict–lossy–legacy–ref | 512 | 0 | 128 | 192 | 192 | 0 |
| Total | | 5,120 | 1,024 | 2,304 | 960 | 1,344 | 512 |

Table 3: C1 outcomes by constructed credential stratum. Each stratum has 768 designed attempts over all routes and repetitions. Labels F0–F7 are defined in the text.

| Stratum | Imported | Rejected | P | V | S | NE |
|---|---|---|---|---|---|---|
| F0 basic | 768 | 0 | 768 | 0 | 0 | 0 |
| F1 PRF with UV | 576 | 192 | 0 | 256 | 192 | 128 |
| F2 PRF without UV | 576 | 192 | 0 | 256 | 192 | 128 |
| F3 largeBlob | 640 | 128 | 384 | 0 | 256 | 0 |
| F4 credBlob | 576 | 192 | 0 | 256 | 192 | 128 |
| F5 payment marker | 640 | 128 | 384 | 0 | 256 | 0 |
| F6 combined extensions | 576 | 192 | 0 | 192 | 256 | 128 |
| F7 optional member | 768 | 0 | 768 | 0 | 0 | 0 |
| Total | 5,120 | 1,024 | 2,304 | 960 | 1,344 | 512 |

no complete PASS because their positive browser behavior cannot be established through the CDP injection path even when their serialized representation is retained. The 512 false-reassurance cases occur in largeBlob-bearing strata F3 and F6: 256 in each, with accepted assertions and a failing executed browser behavior oracle. These 512 cases are a subset of the broader semantic-degradation counts, not an additional category to add to Table 1.

The C2 family outcome matters more than its pooled rejection count. Eight unknown-optional-member and eight minor-version-increase cases were imported but assigned NOT_EVALUABLE; a credential-ID collision was rejected in eight cases while its normative class remained ambiguous. C3's 128 failed atomicity and idempotence controls were exactly the two late failure points of the legacy model, 64 each. Because C3 has no process crash or durable storage, its result cannot predict recovery behavior of a password manager.

# 5 Executable follow-up controls

## 5.1 Experimental exchange and browser PRF loss

Python and Node recovered identical CXF documents in both directions and rejected ciphertext alteration. Software PRF evaluators agreed on retained CXF HMAC seeds and detected their removal. This is a functional software control, not a browser ceremony.

A Chromium virtual authenticator created a PRF-enabled passkey. Three source assertions showed a PRF result stable for one salt and different for a second. The runner exported the signing credential via CDP, carried the CXF document through the experimental exchange, and injected the credential into a new virtual authenticator. A fourth assertion authenticated at the destination, while the PRF result for the original salt was absent. CDP did not expose a PRF seed. This is a loss control for that test path, not a finding about a commercial

credential provider.

A Windows 2025 GitHub Actions replay recorded six exchange and nine browser checks as true. A second replay retained four public signatures, the registration public key, challenges, authenticator and client data, and SHA-256 digests of observed PRF outputs. A separate verifier checked four ES256 signatures, RP binding, challenge uniqueness, UV flags, credential identity, and four PRF digest relationships. The archive contains no private key, seed, or raw PRF output. Comparing hashes proves relations among observed outputs, not the HMAC computation from a seed that CDP did not reveal.

### 5.2 Public-draft envelope structure probe

We added a synthetic shape probe for the CXP Working Draft's visible request and response members, offered parameter selection, and the illustrated ZIP layout with an index JWE and a document JWE. The positive fixture passes those structural checks. Three negative responses with a version mismatch, unoffered archive, or malformed payload encoding are rejected. Additional tests reject unexpected ZIP paths and a plain JSON file substituted for a JWE. The fixture uses standard direct-key AES-GCM JWE solely to exercise the container shape. It does not claim that this maps HPKE to JWE. The draft does not give a complete HPKE-to-JWE binding or include its narrative signed-challenge fields in the displayed request and response objects. Consequently JWE authentication under negotiated HPKE, challenge verification, CXF plaintext recovery, and provider interoperability remain NOT_EVALUABLE in this probe [2]. It is a reproducible parsing control, not a CXP conformance or security result.

We compiled pinned Bitwarden Rust [12] and `go-cxp` Go [13] message types in CI and passed both the same synthetic request and response. Both parsed. Rust serialization dropped request `archive` and `mode` and response `archive`; Go retained both `archive` fields but dropped request `mode`. Both rejected 256 because their version types are eight-bit; 256 fits the draft field width but is not a published version. The dropped fields are type-parser gaps, not encrypted transfers or provider interoperability.

Project Python and Node recovered synthetic CXF from a private two-member HPKE/JWE ZIP (nine local and four cross-runtime controls). Independent Go `jwx/v4` exchanged `HPKE-3-KE` JWE with Python in both directions (five controls), then decrypted our two-member CXP-shaped ZIP (five controls) [16,17]. Both ZIP bindings remain project-authored. Proton Pass common 2.0.0 [15] and Python exchanged direct-HPKE responses in both directions. Credential ID, key, and HMAC seeds survived; four software assertions and four PRF checks passed on the Proton-to-Python path. Proton's direct payload lacks the draft's required ZIP/JWE structure [2], and no provider vault import or deployed RP assertion was tested.

### 5.3 Selected provider case: Fenko

We selected Fenko Vault 0.12.0 [14] after inspecting its CXF path. Two fresh profiles per repetition imported synthetic basic and PRF-bearing credentials. The credentials signed valid assertions. A public verifier rechecked 16 signatures and six PRF observation bindings. The source CXF HMAC seeds were not retained in the observed import/export path; the browser PRF result differed from an oracle derived from those source seeds.

A same-provider round trip between two Fenko profiles produced matching PRF outputs in two selected repetitions. A separate verifier recalculated 24 signatures and 12 PRF bindings. This continuity does not imply that the original seeds survived: this code path derived a value from the imported signing key. Functional equality and preservation of the represented secret are separate claims. Two repetitions are not a product-level reliability estimate.

### 5.4 Positive software control: Proton

The Proton Pass common 2.0.0 software authenticator [15] provided a selected positive control. In two exploratory repetitions, the runner imported CXF, exported it, and reimported into a second software authenticator. Both HMAC seeds survived. With user verification, the expected PRF was reproduced for two salts before and after reimport. A modified-seed control changed the result while the credential still signed. The public verifier rechecked 12 signatures and 12 PRF digest comparisons, including four modified-seed controls. The isolated report cannot reconstruct the HMAC from a withheld seed; that requires repeating the pinned software run. This route used pinned WASM, not a browser passkey provider or the installed Proton Pass application; no no-UV ceremony was run.

## 6 Discussion and limitations

The browser loss control and Fenko case show different ways a signing credential can outlive its original PRF state. The Proton software route shows that preserving CXF seeds and reproducing the expected PRF is possible in one selected implementation. These cases are not a representative sample of provider behavior.

The public CXP wire format was not exchanged between two independent providers. Android's provider-events flow carries CXF JSON through a content URI [18], so a mobile transfer would test a different path. The human-reported Bitwarden-to-Google transfer lacks an instrumented destination assertion and receipt and is excluded from measured results. CDP cannot inject original PRF state; positive browser-to-browser restoration of CXF HMAC seeds is NOT_EVALUABLE. The Proton module result cannot fill that gap. The transaction simulation cannot support claims

about storage durability, and the synthetic campaign cannot support product prevalence or superiority claims.

## 7 Conclusion

PasskeyTransit uses distinct oracles for identity, signing, extensions, and transaction semantics, marking properties an interface cannot expose. The synthetic campaign and selected controls show valid authentication alongside changed or missing properties. Public replays verify observed signatures; provider interoperability and mobile transfer remain open.

## A Open Science

The staged archive contains source, synthetic rows, CXP typeparser and ZIP/JWE reports, and Phase 14 and Fenko/Proton replays. Verifiers recompute 96 C1 cells, hashes, signatures, and Phase 14 report equality. A clean extraction passed on the authors' host, without independent replication or external CI attestation of provider replays. Vaults, private keys, original PRF seeds, and raw PRF outputs are excluded. GitHub provenance remains private; an anonymous URL is pending.

## B Ethical considerations

All reported software credentials and RP accounts were synthetic; the browser control used virtual authenticators and a localhost RP. No real passkey or private vault is distributed. The excluded Android observation is not independently verified. Provider findings apply to tested routes and revisions only.

## References


[1] FIDO Alliance. Credential Exchange Format v1.0 Proposed Standard Errata, 2026. https://fidoalliance.org/specs/cx/cxf-v1.0-ps-errata-20260309.html.

[2] FIDO Alliance. Credential Exchange Protocol v1.0 Working Draft, 3 October 2024. https://fidoalliance.org/specs/cx/cxp-v1.0-wd-20241003.html.

[3] R. Barnes et al. Hybrid Public Key Encryption. RFC 9180, 2022. https://www.rfc-editor.org/rfc/rfc9180.

[4] M. Jones and J. Hildebrand. JSON Web Encryption. RFC 7516, 2015. https://www.rfc-editor.org/rfc/rfc7516.

[5] W3C. Web Authentication Level 3, Recommendation, 25 August 2026. https://www.w3.org/TR/2026/REC-webauthn-3-20260825/.

[6] B. Ramat, D. Kartchner, M. Clark, and K. Seamons. Passkeys in the Wild: A Systematic Study of FIDO2 User Experience Consistency Across Websites. SOUPS, pp. 21–39, 2026. https://www.usenix.org/conference/soups2026/presentation/ramat.

[7] L. Jannett, A. Mayer, M. Westers, V. Mladenov, C. Mainka, and J. Schwenk. The State of Passkeys: Studying the Adoption and Security of Passkeys on the Web. USENIX Security, pp. 5039–5058, 2026. https://www.usenix.org/conference/usenixsecurity26/presentation/jannett.

[8] A. Büttner and N. Gruschka. Device-Bound vs. Synced Credentials: A Comparative Evaluation of Passkey Authentication. ICISSP, 2025. https://arxiv.org/abs/2501.07380.

[9] A. Daffalla et al. A Framework for Abusability Analysis: The Case of Passkeys in Interpersonal Threat Models. USENIX Security, pp. 7819–7838, 2025. https://www.usenix.org/conference/usenixsecurity25/presentation/daffalla.

[10] M. Islam et al. Detecting Compromise of Passkey Storage on the Cloud. USENIX Security, pp. 7743–7762, 2025. https://www.usenix.org/conference/usenixsecurity25/presentation/islam.

[11] P. Henarejos. Vaulted Passkeys: A Device-Bound Proposal for Authenticated Credential Export and Import. arXiv:2608.13806, 2026. https://arxiv.org/abs/2608.13806.

[12] Bitwarden. Credential Exchange Protocol Rust types, revision `32da4c8`, 2026. https://github.com/bitwarden/credential-exchange.

[13] N. Vinuesa. `go-cxp`, revision `94659f3`, 2026. https://github.com/nvinuesa/go-cxp.

[14] FenkoHQ. Passkey Vault 0.12.0, revision `2d873fb`. https://github.com/FenkoHQ/passkey-vault.

[15] Proton. Proton Pass common 2.0.0; pinned revisions in artifact. https://github.com/protonpass/proton-pass-common.

[16] T. Reddy et al. Use of HPKE with JWE, Internet-Draft `draft-ietf-jose-hpke-encrypt-22`, 2026. https://datatracker.ietf.org/doc/draft-ietf-jose-hpke-encrypt/22/.

[17] lestrrat-go. `jwx/v4`, v4.6.0, 2026. https://github.com/lestrrat-go/jwx/tree/v4.6.0.

[18] Android Developers. Credential transfer, 2026. https://developer.android.com/identity/sign-in/credential-transfer.